

# Skylight Calendar – Complete Documentation

A Raspberry Pi-powered digital calendar display

Skylight Calendar Project

June 2026

## Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Introduction</b>                                 | <b>4</b>  |
| What is Skylight Calendar? . . . . .                | 4         |
| Key Features . . . . .                              | 4         |
| System Requirements . . . . .                       | 4         |
| <b>Architecture Overview</b>                        | <b>5</b>  |
| System Diagram . . . . .                            | 5         |
| Technology Stack . . . . .                          | 5         |
| Directory Structure . . . . .                       | 5         |
| <b>Setup Guide</b>                                  | <b>7</b>  |
| Step 1: Google Cloud Configuration . . . . .        | 7         |
| Create a Google Cloud Project . . . . .             | 7         |
| Enable the Calendar API . . . . .                   | 7         |
| Create OAuth 2.0 Credentials . . . . .              | 7         |
| Important Note on Consent Screen . . . . .          | 7         |
| Step 2: OpenWeatherMap Setup . . . . .              | 7         |
| Step 3: Configure Environment . . . . .             | 8         |
| Step 4: Install and Run (Development) . . . . .     | 8         |
| Step 5: Deploy to Raspberry Pi . . . . .            | 8         |
| Transfer the project . . . . .                      | 8         |
| Run the setup script . . . . .                      | 8         |
| First-time authentication . . . . .                 | 9         |
| <b>Usage Examples</b>                               | <b>10</b> |
| Example 1: Family Calendar . . . . .                | 10        |
| Example 2: Office Conference Room Display . . . . . | 10        |
| Example 3: Personal Dashboard . . . . .             | 10        |
| <b>API Reference</b>                                | <b>11</b> |
| Authentication Endpoints . . . . .                  | 11        |
| GET /api/auth/status . . . . .                      | 11        |
| GET /api/auth/login . . . . .                       | 11        |

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| GET /api/auth/callback . . . . .                                    | 11        |
| POST /api/auth/logout . . . . .                                     | 11        |
| Calendar Endpoints . . . . .                                        | 11        |
| GET /api/calendar/list . . . . .                                    | 11        |
| GET /api/calendar/events . . . . .                                  | 12        |
| Weather Endpoint . . . . .                                          | 12        |
| GET /api/weather/current . . . . .                                  | 12        |
| <b>Running Tests</b>                                                | <b>14</b> |
| Server Tests . . . . .                                              | 14        |
| Client Tests . . . . .                                              | 14        |
| Running All Tests . . . . .                                         | 14        |
| <b>Customization</b>                                                | <b>15</b> |
| Changing the Color Scheme . . . . .                                 | 15        |
| Dark Mode . . . . .                                                 | 15        |
| Changing the Week Start Day . . . . .                               | 15        |
| Adjusting Refresh Intervals . . . . .                               | 15        |
| Portrait Mode Display . . . . .                                     | 16        |
| <b>Frequently Asked Questions</b>                                   | <b>17</b> |
| General . . . . .                                                   | 17        |
| Calendar . . . . .                                                  | 17        |
| Weather . . . . .                                                   | 18        |
| Raspberry Pi . . . . .                                              | 18        |
| <b>Troubleshooting</b>                                              | <b>19</b> |
| Authentication Issues . . . . .                                     | 19        |
| "Not authenticated" after connecting Google Calendar . . . . .      | 19        |
| Google OAuth shows "Access Blocked" or "App not verified" . . . . . | 19        |
| Redirect URI mismatch error . . . . .                               | 19        |
| Display Issues . . . . .                                            | 19        |
| Calendar shows "Loading calendar..." indefinitely . . . . .         | 19        |
| Events appear on the wrong day . . . . .                            | 19        |
| Text is too small on the TV . . . . .                               | 20        |
| White screen / blank page on the TV . . . . .                       | 20        |
| Weather Issues . . . . .                                            | 20        |
| Weather widget not appearing . . . . .                              | 20        |
| Weather shows wrong temperature . . . . .                           | 20        |
| Server Issues . . . . .                                             | 21        |
| Server won't start: "EADDRINUSE" . . . . .                          | 21        |
| Server crashes with "ENOMEM" . . . . .                              | 21        |
| systemd service not starting on boot . . . . .                      | 21        |
| Network Issues . . . . .                                            | 21        |
| Cannot access Pi from another device . . . . .                      | 21        |
| Calendar stops updating . . . . .                                   | 22        |
| <b>Maintenance</b>                                                  | <b>23</b> |

|                                    |           |
|------------------------------------|-----------|
| Updating the Application . . . . . | 23        |
| Backing Up Configuration . . . . . | 23        |
| Monitoring . . . . .               | 23        |
| Automatic Restarts . . . . .       | 23        |
| <b>Security Considerations</b>     | <b>24</b> |

# Introduction

## What is Skylight Calendar?

Skylight Calendar is an open-source digital calendar display designed to run on a Raspberry Pi connected to a TV or monitor via HDMI. It syncs with Google Calendar and presents your schedule in a beautiful, minimal week-view interface inspired by the commercial Skylight Calendar product.

The display is designed to be always-on and glanceable – large typography, color-coded events by calendar, and a clean warm aesthetic that looks great in a kitchen, hallway, or office.

## Key Features

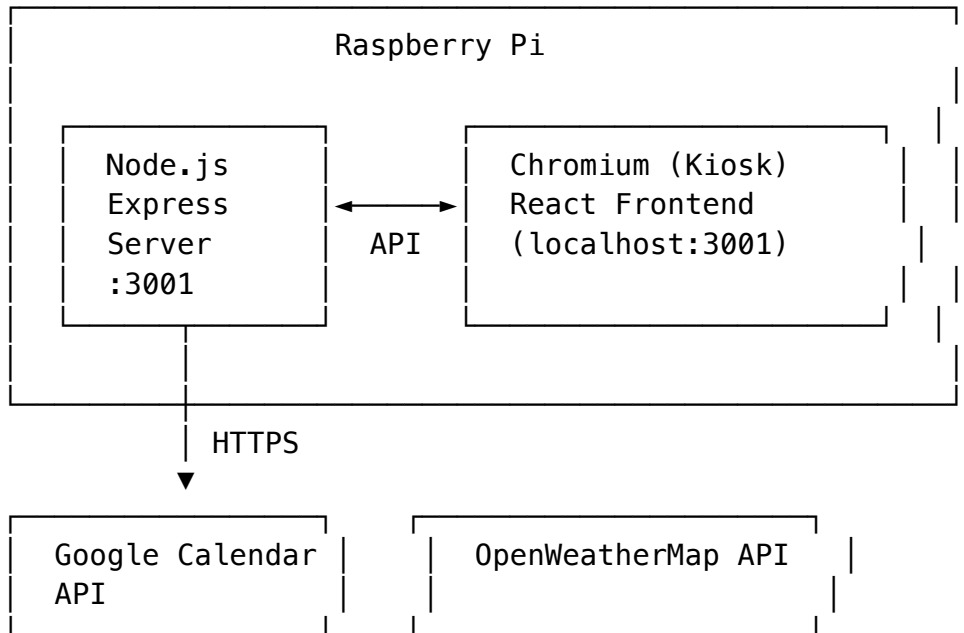
- **Week View Display:** Seven-day grid showing all upcoming events at a glance
- **Multi-Calendar Support:** Sync multiple Google Calendars with distinct colors per calendar (great for families)
- **Weather Widget:** Current temperature and conditions displayed in the header
- **Auto-Refresh:** Events update every 5 minutes, weather every 30 minutes
- **Kiosk Mode:** Full-screen display with no cursor, no screen blanking, auto-start on boot
- **TV-Optimized:** Designed for 1080p displays with large, readable typography
- **Minimal Maintenance:** Once set up, runs unattended indefinitely

## System Requirements

| Component | Requirement                                           |
|-----------|-------------------------------------------------------|
| Hardware  | Raspberry Pi 4 or 5 (2GB+ RAM recommended)            |
| OS        | Raspberry Pi OS with Desktop (Bookworm or later)      |
| Display   | Any TV or monitor with HDMI input (1080p recommended) |
| Network   | WiFi or Ethernet connection                           |
| Software  | Node.js 20+, Chromium browser                         |

# Architecture Overview

## System Diagram



## Technology Stack

| Layer        | Technology         | Purpose                                  |
|--------------|--------------------|------------------------------------------|
| Backend      | Node.js + Express  | API server, OAuth handling, data caching |
| Frontend     | React + Vite       | Calendar UI rendering                    |
| Auth         | Google OAuth 2.0   | Calendar data access                     |
| Weather      | OpenWeatherMap 3.0 | Current conditions + forecast            |
| Display      | Chromium Kiosk     | Full-screen browser on Pi                |
| Process Mgmt | systemd            | Server auto-start on boot                |

## Directory Structure

```
skylight-calendar/
├── .env.example      # Environment variable template
├── .env              # Your configuration (not in git)
├── .tokens.json      # Google OAuth tokens (not in git)
├── package.json      # Root package with orchestration scripts
└── server/
```

```

├── package.json
├── src/
│   ├── index.js      # Express server entry point
│   └── routes/
│       ├── auth.js   # Google OAuth flow
│       ├── calendar.js # Calendar list + events
│       └── weather.js # Weather data proxy
├── tests/             # Server unit tests
├── client/
│   ├── package.json
│   ├── index.html
│   ├── src/
│   │   ├── main.jsx    # React entry point
│   │   ├── App.jsx     # Root component
│   │   ├── styles/
│   │   │   └── global.css # CSS variables and base styles
│   │   ├── components/
│   │   │   ├── Header.jsx
│   │   │   ├── WeekView.jsx
│   │   │   ├── DayColumn.jsx
│   │   │   ├── EventCard.jsx
│   │   │   ├── WeatherWidget.jsx
│   │   │   └── SetupScreen.jsx
│   │   └── hooks/
│   │       ├── useCalendarData.js
│   │       └── useWeatherData.js
│   └── src/__tests__/  # Client unit tests
├── scripts/
│   ├── setup-pi.sh     # Automated Pi configuration
│   └── start-kiosk.sh  # Generated kiosk launcher
├── docs/
│   └── skylight-calendar-documentation.md

```

# Setup Guide

## Step 1: Google Cloud Configuration

### Create a Google Cloud Project

1. Navigate to the Google Cloud Console: <https://console.cloud.google.com/>
2. Click **Select a project** → **New Project**
3. Name it (e.g., “Skylight Calendar”) and click **Create**
4. Select the newly created project

### Enable the Calendar API

1. Go to **APIs & Services** → **Library**
2. Search for “Google Calendar API”
3. Click on it and press **Enable**

### Create OAuth 2.0 Credentials

1. Go to **APIs & Services** → **Credentials**
2. Click + **CREATE CREDENTIALS** → **OAuth client ID**
3. If prompted, configure the consent screen:
  - User Type: **External**
  - App name: “Skylight Calendar”
  - Add your email as a test user
4. Application type: **Web application**
5. Name: “Skylight Calendar”
6. Under **Authorized redirect URIs**, add:
  - <http://localhost:3001/api/auth/callback> (for development)
  - <http://<your-pi-ip>:3001/api/auth/callback> (for production)
7. Click **Create** and note the **Client ID** and **Client Secret**

### Important Note on Consent Screen

If your app is in “Testing” mode, only users you explicitly add as test users can authenticate. You can add up to 100 test users without going through Google’s verification process.

## Step 2: OpenWeatherMap Setup

1. Create an account at <https://openweathermap.org/>
2. Go to **API keys** in your account dashboard
3. Subscribe to **One Call API 3.0** (requires payment method but includes 1,000 free calls/day – more than enough for this use case)
4. Copy your API key
5. Find your location coordinates:
  - Open Google Maps
  - Right-click on your location

- The first line shows latitude, longitude (e.g., 40.7128, -74.0060)

### Step 3: Configure Environment

```
cd skylight-calendar
cp .env.example .env
```

Edit .env with your values:

```
GOOGLE_CLIENT_ID=123456789.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=GOCSPX-xxxxxxxxxxxxx
GOOGLE_REDIRECT_URI=http://localhost:3001/api/auth/callback
OPENWEATHER_API_KEY=abcdef1234567890
WEATHER_LAT=40.7128
WEATHER_LON=-74.0060
PORT=3001
```

### Step 4: Install and Run (Development)

```
# Install all dependencies
```

```
npm install
cd server && npm install && cd ..
cd client && npm install && cd ..
```

```
# Start in development mode (both server and client with hot-reload)
```

```
npm run dev
```

Open <http://localhost:5173> in your browser. You'll see the setup screen prompting you to connect Google Calendar.

### Step 5: Deploy to Raspberry Pi

#### Transfer the project

```
# From your development machine
```

```
scp -r skylight-calendar/ pi@<pi-ip>:~/skylight-calendar/
```

Or clone from a git repository:

```
ssh pi@<pi-ip>
git clone <your-repo-url> ~/skylight-calendar
```

#### Run the setup script

```
cd ~/skylight-calendar
cp .env.example .env
nano .env # Fill in your credentials
bash scripts/setup-pi.sh
sudo reboot
```



## **First-time authentication**

After the Pi reboots:

1. From any device on the same network, open a browser to `http://<pi-ip>:3001`
2. Click "Connect Google Calendar"
3. Complete the Google sign-in flow
4. The TV display will automatically start showing your calendar

## Usage Examples

### Example 1: Family Calendar

A family of four, each with their own Google Calendar, wants a shared display in the kitchen.

#### Setup:

1. Create one Google account to act as the “display” account
2. Have each family member share their calendar with this account:
  - In Google Calendar → Settings → Share with specific people
  - Add the display account email with “See all event details” permission
3. Connect the display account to Skylight Calendar
4. All shared calendars appear automatically with distinct colors

**Result:** The kitchen TV shows everyone’s schedule – work meetings, school events, soccer practice, and date nights – each color-coded by family member.

### Example 2: Office Conference Room Display

Mount a TV outside a conference room to show the room’s availability.

#### Setup:

1. Connect the room’s resource calendar (your Google Workspace admin can share it)
2. Configure only that one calendar in the system
3. The display shows when the room is free vs. booked

### Example 3: Personal Dashboard

Use as a personal scheduling dashboard in a home office.

#### Setup:

1. Connect your primary Google account
2. Enable your work calendar, personal calendar, and any subscribed calendars (holidays, sports)
3. Add weather for your location
4. Mount a small monitor on your desk or wall

## API Reference

### Authentication Endpoints

#### GET /api/auth/status

Returns the current authentication state.

##### Response:

```
{ "authenticated": true }
```

#### GET /api/auth/login

Redirects the user to Google's OAuth consent screen.

#### GET /api/auth/callback

OAuth callback handler. Google redirects here after the user grants permission. Stores tokens and shows a success message.

#### POST /api/auth/logout

Removes stored authentication tokens.

##### Response:

```
{ "success": true }
```

### Calendar Endpoints

#### GET /api/calendar/list

Returns all calendars accessible to the authenticated user.

##### Response:

```
[
  {
    "id": "primary",
    "summary": "John's Calendar",
    "color": "#4285f4",
    "primary": true
  },
  {
    "id": "family@group.calendar.google.com",
    "summary": "Family",
    "color": "#33b679",
    "primary": false
  }
]
```

## GET /api/calendar/events

Returns events for the specified calendars and time range.

### Query Parameters:

| Parameter | Type       | Description                                             |
|-----------|------------|---------------------------------------------------------|
| calendars | string     | Comma-separated calendar IDs                            |
| timeMin   | ISO string | Start of time range (defaults to start of current week) |
| timeMax   | ISO string | End of time range (defaults to end of current week)     |

### Response:

```
[
  {
    "id": "abc123",
    "calendarId": "primary",
    "title": "Team Standup",
    "start": "2026-06-28T09:00:00-04:00",
    "end": "2026-06-28T09:30:00-04:00",
    "allDay": false,
    "location": "Conference Room B",
    "description": "Daily sync meeting"
  }
]
```

## Weather Endpoint

### GET /api/weather/current

Returns current weather and 7-day forecast.

### Response:

```
{
  "current": {
    "temp": 72,
    "condition": "Clear",
    "icon": "01d",
    "description": "clear sky"
  },
  "daily": [
    {
      "date": "2026-06-28T00:00:00.000Z",
      "high": 85,
      "low": 68,
      "condition": "Clear",

```

```
    "icon": "01d"  
  }  
]  
}
```

## Running Tests

### Server Tests

```
cd server
npm test           # Run all tests once
npm run test:watch # Run in watch mode during development
```

Server tests cover:

- **Authentication routes:** OAuth flow, token persistence, login/logout
- **Calendar routes:** Calendar listing, event fetching, multi-calendar merging, error handling
- **Weather routes:** API integration, data transformation, error states, response caching

### Client Tests

```
cd client
npm test           # Run all tests once
npm run test:watch # Run in watch mode during development
```

Client tests cover:

- **EventCard:** Rendering, time formatting, all-day events, color application
- **DayColumn:** Day header, event listing, today highlighting, empty state
- **WeekView:** 7-day grid, event distribution, loading state
- **WeatherWidget:** Condition icons, temperature display, null handling
- **SetupScreen:** Connect button, instructional text

### Running All Tests

From the project root:

```
cd server && npm test && cd ../client && npm test
```

## Customization

### Changing the Color Scheme

Edit `client/src/styles/global.css`:

```
:root {
  --bg-primary: #faf8f5;      /* Page background */
  --bg-secondary: #ffffff;    /* Card background */
  --text-primary: #1a1a2e;    /* Main text */
  --text-secondary: #4a5568;  /* Secondary text */
  --text-muted: #a0aec0;      /* Muted/hint text */
  --border: #e8e4df;          /* Borders */
  --today-highlight: #fff9e6; /* Today column background */
}
```

### Dark Mode

Replace the CSS variables with dark-mode values:

```
:root {
  --bg-primary: #1a1a2e;
  --bg-secondary: #2d2d44;
  --text-primary: #e2e8f0;
  --text-secondary: #a0aec0;
  --text-muted: #718096;
  --border: #4a5568;
  --today-highlight: #2d3748;
}
```

### Changing the Week Start Day

To start the week on Monday instead of Sunday, modify the `startOfWeek` function in both:

- `server/src/routes/calendar.js`
- `client/src/hooks/useCalendarData.js`

Change:

```
d.setDate(d.getDate() - d.getDay());
```

To:

```
const day = d.getDay();
d.setDate(d.getDate() - (day === 0 ? 6 : day - 1));
```

### Adjusting Refresh Intervals

In `client/src/hooks/useCalendarData.js`:

```
const interval = setInterval(load, 5 * 60 * 1000); // Change 5 to desired minutes
```

In `client/src/hooks/useWeatherData.js`:

```
const interval = setInterval(fetchWeather, 30 * 60 * 1000); // Change 30 to desired
```

## Portrait Mode Display

For a vertically-mounted monitor, change the grid in `client/src/components/WeekView.j`

```
grid: {  
  display: 'grid',  
  gridTemplateColumns: '1fr', // Stack days vertically  
  gridTemplateRows: 'repeat(7, 1fr)',  
  gap: '8px',  
  flex: 1,  
}
```



## Frequently Asked Questions

### General

**Q: Does this require a constant internet connection?**

A: Yes, for syncing calendar events and fetching weather data. However, if the connection drops temporarily, the last-fetched data remains displayed. The app will resume syncing when connectivity returns.

**Q: Can I use this without a Raspberry Pi?**

A: Yes. Any computer that can run Node.js and Chromium will work. The kiosk setup script is Pi-specific, but the app itself is platform-agnostic. You could run it on an old laptop connected to a TV.

**Q: How much does this cost to run?**

A: Hardware cost is a Raspberry Pi (\$35–80) plus any monitor/TV you already have. Operating costs are essentially zero – Google Calendar API and OpenWeatherMap free tier are sufficient for a single display.

**Q: Can multiple TVs share the same backend?**

A: Yes. Run one server instance and point multiple browser displays at `http://<server-ip>:3001`. All will show the same calendar data.

### Calendar

**Q: How do I add or remove calendars from the display?**

A: The system displays all calendars shared with the connected Google account. To control visibility:

- **Add a calendar:** Share it with the account connected to Skylight Calendar
- **Remove a calendar:** Unshare it, or hide it in Google Calendar settings for that account

**Q: Can I show calendars from different Google accounts?**

A: The simplest approach is to share calendars from other accounts to the one connected to Skylight Calendar. Direct multi-account OAuth is not currently supported.

**Q: Why don't I see my events?**

A: Check these common causes:

1. The calendar might not be shared with the connected account
2. The OAuth token might have expired – visit `http://<pi-ip>:3001/api/auth/status` to check
3. Events might be outside the current week's time range

**Q: Does it support recurring events?**

A: Yes. The Google Calendar API expands recurring events into individual instances via the `singleEvents: true` parameter. Each occurrence appears as its own event on the correct day.

## Weather

**Q: Can I use Celsius instead of Fahrenheit?**

A: Change `units=imperial` to `units=metric` in `server/src/routes/weather.js`:

```
const url = `...&units=metric&...`;
```

**Q: The weather widget shows nothing. What's wrong?**

A: Verify your OpenWeatherMap API key is active and that you've subscribed to the One Call API 3.0. New API keys can take a few minutes to activate.

## Raspberry Pi

**Q: Can I use a Pi Zero 2 W?**

A: It's possible but not recommended. The Pi Zero 2 W has limited RAM (512MB) which makes running Chromium unreliable. Consider using a lightweight static page approach instead of a full React app if you must use a Zero.

**Q: My Pi screen goes black after a while. How do I fix this?**

A: The setup script disables screen blanking, but if it's still happening:

```
# Disable screen blanking manually
xset s off
xset -dpms
xset s noblank
```

Add these to `/etc/xdg/lxsession/LXDE-pi/autostart` to persist across re-boots.

**Q: How do I update the app after making changes?**

A: SSH into the Pi and:

```
cd ~/skylight-calendar
git pull # if using git
cd client && npm run build && cd ..
sudo systemctl restart skylight-calendar
```

# Troubleshooting

## Authentication Issues

### “Not authenticated” after connecting Google Calendar

**Cause:** OAuth tokens have expired and cannot be refreshed.

**Fix:**

*# Remove stale tokens*

*rm .tokens.json*

*# Re-authenticate by visiting the setup screen*

*# Navigate to <http://<pi-ip>:3001> from another device*

### Google OAuth shows “Access Blocked” or “App not verified”

**Cause:** Your Google Cloud project is in testing mode and the user is not added as a test user.

**Fix:** 1. Go to Google Cloud Console → APIs & Services → OAuth consent screen 2. Under “Test users”, add the Google account you’re trying to connect 3. Try the login flow again

## Redirect URI mismatch error

**Cause:** The redirect URI in your .env doesn’t match what’s configured in Google Cloud Console.

**Fix:** Ensure GOOGLE\_REDIRECT\_URI in .env exactly matches one of the “Authorized redirect URIs” in your OAuth client configuration. Common mistakes: – Trailing slash mismatch (/callback vs /callback/) – localhost vs actual IP address – Wrong port number

## Display Issues

### Calendar shows “Loading calendar...” indefinitely

**Cause:** The server cannot reach Google’s API, or the frontend cannot reach the backend.

**Fix:** 1. Check server logs: `journalctl -u skylight-calendar -f` 2. Test the API directly: `curl http://localhost:3001/api/auth/status` 3. Verify network connectivity: `ping google.com`

### Events appear on the wrong day

**Cause:** Timezone mismatch between the Pi and your Google Calendar.

**Fix:**

```
# Set the correct timezone on the Pi
sudo raspi-config
# Navigate to: Localisation Options → Timezone
```

### **Text is too small on the TV**

**Cause:** The Pi might be outputting at a non-native resolution.

**Fix:**

```
# Check current resolution
xrandr

# Force 1080p output (edit /boot/config.txt)
hdmi_group=1
hdmi_mode=16
```

Alternatively, adjust font sizes in `client/src/styles/global.css` and component style objects.

### **White screen / blank page on the TV**

**Cause:** The client build is missing or the server can't serve static files.

**Fix:**

```
cd ~/skylight-calendar/client
npm run build
# Verify dist/ directory exists and contains index.html
ls dist/
```

## **Weather Issues**

### **Weather widget not appearing**

**Cause:** Missing or invalid OpenWeatherMap configuration.

**Fix:** 1. Verify `.env` has all three weather values set: – `OPENWEATHER_API_KEY` – `WEATHER_LAT` – `WEATHER_LON` 2. Test the endpoint: `curl http://localhost:3001/api/weather` 3. Check if your API key is active at [https://home.openweathermap.org/api\\_keys](https://home.openweathermap.org/api_keys)

### **Weather shows wrong temperature**

**Cause:** Incorrect coordinates or units.

**Fix:** Verify your latitude/longitude values are correct. Positive latitude = North, negative = South. Positive longitude = East, negative = West. For the US, longitude should be negative (e.g., `-74.006` for New York).

## Server Issues

### Server won't start: "EADDRINUSE"

**Cause:** Another instance is already running on port 3001.

**Fix:**

*# Find and kill the existing process*

```
lsof -i :3001
```

```
kill <PID>
```

*# Or change the port in .env*

```
PORT=3002
```

### Server crashes with "ENOMEM"

**Cause:** Raspberry Pi running out of memory.

**Fix:**

*# Add swap space*

```
sudo dphys-swapfile swapoff
```

```
sudo nano /etc/dphys-swapfile # Set CONF_SWAPSIZE=512
```

```
sudo dphys-swapfile setup
```

```
sudo dphys-swapfile swapon
```

### systemd service not starting on boot

**Fix:**

*# Check service status*

```
sudo systemctl status skylight-calendar
```

*# View recent logs*

```
journalctl -u skylight-calendar --since "5 min ago"
```

*# Re-enable the service*

```
sudo systemctl enable skylight-calendar
```

```
sudo systemctl start skylight-calendar
```

## Network Issues

### Cannot access Pi from another device

**Cause:** Firewall or network isolation.

**Fix:**

*# Check if the server is listening*

```
ss -tlnp | grep 3001
```

```
# Verify the Pi's IP address  
hostname -I
```

```
# Ensure devices are on the same network/subnet
```

### **Calendar stops updating**

**Cause:** Google OAuth refresh token expired (rare, but happens if the app is unused for 6+ months or the account password changes).

**Fix:** Re-authenticate by visiting the setup screen. The app will obtain a new refresh token.

## Maintenance

### Updating the Application

```
ssh pi@<pi-ip>  
cd ~/skylight-calendar  
git pull origin main  
cd client && npm run build && cd ..  
sudo systemctl restart skylight-calendar
```

### Backing Up Configuration

The only files you need to back up are: - .env (your API credentials) - .tokens.json (Google OAuth tokens)

```
cp .env .env.backup  
cp .tokens.json .tokens.json.backup
```

### Monitoring

Check the server status:

```
sudo systemctl status skylight-calendar
```

View live logs:

```
journalctl -u skylight-calendar -f
```

### Automatic Restarts

The systemd service is configured to restart on failure:

```
Restart=on-failure  
RestartSec=5
```

If the server crashes, it will automatically restart within 5 seconds.

## Security Considerations

- **OAuth tokens** are stored locally in `.tokens.json`. Keep this file secure and never commit it to version control.
- **API keys** are stored in `.env`. Same guidance applies.
- The server binds to `0.0.0.0` (all interfaces) so it's accessible from your local network. Do **not** expose port `3001` to the public internet.
- If you must access the calendar remotely, use a VPN or SSH tunnel rather than opening the port in your router.
- Google OAuth tokens include a refresh token that grants ongoing access to your calendar data. If the Pi is compromised, revoke access at <https://myaccount.google.com/permissions>.